

CSS Flexbox Introduction

Complete Guide: From Basics to Advanced

Contents

1	Introduction	2
1.1	What is Flexbox?	2
1.2	Why Use Flexbox?	2
1.3	Flexbox vs. Grid vs. Float	3
1.4	Browser Support	3
2	Core Flexbox Concepts	4
2.1	Flex Container and Flex Items	4
2.1.1	The Flex Container	4
2.1.2	Flex Items	4
2.2	Main Axis and Cross Axis	4
2.2.1	Main Axis	4
2.2.2	Cross Axis	4
2.3	Key Flexbox Properties Overview	5
2.3.1	Container Properties	5
2.3.2	Item Properties	5
3	Container Properties	6
3.1	flex-direction	6
3.1.1	Values	6
3.1.2	Examples	6
3.2	justify-content	6
3.2.1	Values	7
3.2.2	Examples	7
3.3	align-items	7
3.3.1	Values	7
3.3.2	Examples	8
3.4	Perfect Centering with Flexbox	8
3.5	flex-wrap	8
3.5.1	Values	8
3.5.2	Examples	8
3.6	gap	9
4	Item Properties	10
4.1	flex-grow	10
4.1.1	Values	10
4.1.2	Examples	10

4.2	flex-shrink	11
4.2.1	Values	11
4.2.2	Examples	11
4.3	flex-basis	11
4.3.1	Values	11
4.3.2	Examples	12
4.4	flex Shorthand	12
4.5	order	12
4.6	align-self	13
5	Practical Examples	14
5.1	Example 1: Navigation Bar	14
5.2	Example 2: Card Layout	15
5.3	Example 3: Form Layout	16
5.4	Example 4: Footer Layout	17
6	Responsive Flexbox	19
6.1	Mobile-First Approach	19
6.2	Responsive Card Grid	20
7	Common Issues & Solutions	21
7.1	Issue 1: Items Not Shrinking Below Content Size	21
7.2	Issue 2: Last Item Takes Extra Space	21
7.3	Issue 3: Content Overflows Container	21
7.4	Issue 4: Unequal Item Sizes	22
8	Best Practices	23
8.1	Code Organization	23
8.2	Accessibility Tips	23
8.3	Performance Tips	24
8.4	When to Use Flexbox	24
8.5	When NOT to Use Flexbox	24
9	Summary: Key Takeaways	25
9.1	Core Concepts	25
9.2	Essential Properties	25
9.3	Quick Reference	25
9.4	Alignment Cheat Sheet	26
9.5	Key Rules	26

Chapter 1

Introduction

1.1 What is Flexbox?

Flexbox (Flexible Box Layout) is a CSS layout module that provides an efficient way to arrange, distribute, and align elements within a container. It automatically adjusts element sizes and spacing to best fill available space, making it perfect for responsive designs.

Key Point: Flexbox is one-dimensional. It works best for layouts in a single direction (row or column), while CSS Grid handles two-dimensional layouts.

1.2 Why Use Flexbox?

- Create responsive layouts without media queries
- Easily center and align content
- Distribute space evenly between items
- Build navigation bars and menus
- Create flexible card layouts
- Handle flexible sizing automatically
- Simple and intuitive syntax
- Excellent browser support

1.3 Flexbox vs. Grid vs. Float

Flexbox	CSS Grid	Float
One-dimensional Simple layouts	Two-dimensional Complex layouts	Legacy method Not recommended
Content-driven	Layout-driven	Causes issues
Easy alignment	Precise control	Hard to center
Modern support	Modern support	Old support

1.4 Browser Support

Flexbox has excellent support in all modern browsers:

- Chrome 29+
- Firefox 28+
- Safari 9+
- Edge 12+
- Opera 12.1+
- IE 11+ (partial support)

Chapter 2

Core Flexbox Concepts

2.1 Flex Container and Flex Items

2.1.1 The Flex Container

The flex container is the parent element with `display: flex`. It controls how all child elements (flex items) are arranged.

```
.flex-container {  
    display: flex; /* Enables flexbox */  
}
```

2.1.2 Flex Items

Flex items are direct children of the flex container. They automatically become flexible and respond to flex properties.

```
<div class="flex-container">  
    <div class="flex-item">Item 1</div>  
    <div class="flex-item">Item 2</div>  
    <div class="flex-item">Item 3</div>  
</div>
```

2.2 Main Axis and Cross Axis

Understanding axes is crucial to flexbox.

2.2.1 Main Axis

The main axis is the primary axis along which flex items are laid out. By default, it runs left to right (horizontal).

2.2.2 Cross Axis

The cross axis is perpendicular to the main axis. By default, it runs top to bottom (vertical).

```
/* Default: Main axis = horizontal, Cross axis = vertical */
.flex-container {
  display: flex;
  flex-direction: row; /* Main axis: left to right */
}

/* Changed: Main axis = vertical, Cross axis = horizontal */
.flex-container {
  display: flex;
  flex-direction: column; /* Main axis: top to bottom */
}
```

2.3 Key Flexbox Properties Overview

2.3.1 Container Properties

- `display: flex` - Enable flexbox
- `flex-direction` - Direction of main axis
- `justify-content` - Align along main axis
- `align-items` - Align along cross axis
- `flex-wrap` - Wrap items to new lines
- `gap` - Space between items
- `align-content` - Align wrapped lines

2.3.2 Item Properties

- `flex-grow` - Grow to fill space
- `flex-shrink` - Shrink when space is limited
- `flex-basis` - Base size before growing/shrinking
- `flex` - Shorthand for grow, shrink, basis
- `order` - Visual order of items
- `align-self` - Override align-items for item

Chapter 3

Container Properties

3.1 flex-direction

Controls the direction of the main axis.

3.1.1 Values

- `row` - Items flow left to right (default)
- `row-reverse` - Items flow right to left
- `column` - Items flow top to bottom
- `column-reverse` - Items flow bottom to top

3.1.2 Examples

```
/* Horizontal layout (default) */  
.flex-row {  
    display: flex;  
    flex-direction: row;  
}  
  
/* Vertical layout */  
.flex-column {  
    display: flex;  
    flex-direction: column;  
}  
  
/* Reverse horizontal */  
.flex-row-reverse {  
    display: flex;  
    flex-direction: row-reverse;  
}
```

3.2 justify-content

Aligns items along the main axis (horizontally if row, vertically if column).

3.2.1 Values

- `flex-start` - Items at start (default)
- `flex-end` - Items at end
- `center` - Items centered
- `space-between` - Space between items
- `space-around` - Space around items
- `space-evenly` - Equal space everywhere

3.2.2 Examples

```
/* Center items horizontally */
.flex-center {
    display: flex;
    justify-content: center;
}

/* Spread items with space between */
.flex-spread {
    display: flex;
    justify-content: space-between;
}

/* Items at end */
.flex-end {
    display: flex;
    justify-content: flex-end;
}

/* Equal space everywhere */
.flex-evenly {
    display: flex;
    justify-content: space-evenly;
}
```

3.3 align-items

Aligns items along the cross axis (vertically if row, horizontally if column).

3.3.1 Values

- `stretch` - Items stretch to fill container (default)
- `flex-start` - Items at start
- `flex-end` - Items at end
- `center` - Items centered
- `baseline` - Items aligned to baseline

3.3.2 Examples

```
/* Center items vertically */
.flex-center-vertical {
  display: flex;
  height: 200px;
  align-items: center;
}

/* Items at bottom */
.flex-bottom {
  display: flex;
  height: 200px;
  align-items: flex-end;
}

/* Items stretch to fill height */
.flex-stretch {
  display: flex;
  height: 200px;
  align-items: stretch;
}
```

3.4 Perfect Centering with Flexbox

```
/* Center content both horizontally and vertically */
.perfect-center {
  display: flex;
  justify-content: center; /* Horizontal centering */
  align-items: center;      /* Vertical centering */
  width: 300px;
  height: 300px;
}
```

3.5 flex-wrap

Controls whether items wrap to new lines or shrink.

3.5.1 Values

- nowrap - All items on one line (default)
- wrap - Items wrap to new lines
- wrap-reverse - Items wrap in reverse

3.5.2 Examples

```
/* Allow items to wrap */
.flex-wrap {
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
}

/* All items on one line (may shrink) */
.flex-nowrap {
  display: flex;
  flex-wrap: nowrap;
}

/* Wrap in reverse order */
.flex-wrap-reverse {
  display: flex;
  flex-wrap: wrap-reverse;
}
```

3.6 gap

Adds space between flex items.

```
/* Gap between all items */
.flex-gap {
  display: flex;
  gap: 20px; /* 20px gap between items */
}

/* Different row and column gaps */
.flex-gap-custom {
  display: flex;
  row-gap: 10px;
  column-gap: 20px;
}
```

Chapter 4

Item Properties

4.1 flex-grow

Controls how much an item grows to fill available space.

4.1.1 Values

- 0 - Don't grow (default)
- 1 - Grow equally with other items
- 2 - Grow twice as much
- Any positive number

4.1.2 Examples

```
/* All items grow equally */
.flex-item {
    flex-grow: 1;
}

/* One item grows more */
.flex-item {
    flex-grow: 1;
}

.flex-item.featured {
    flex-grow: 2; /* Takes double space */
}

/* Create auto-layout columns */
.flex-column-layout {
    display: flex;
    gap: 20px;
}

.sidebar {
    flex-grow: 0;
```

```
        width: 250px;
    }

    .main-content {
        flex-grow: 1; /* Takes remaining space */
    }
```

4.2 flex-shrink

Controls how much an item shrinks when space is limited.

4.2.1 Values

- 1 - Shrink equally (default)
- 0 - Don't shrink
- Higher numbers shrink more aggressively

4.2.2 Examples

```
/* Don't shrink this item */
.no-shrink {
    flex-shrink: 0;
    width: 200px;
}

/* Shrink aggressively */
.aggressive-shrink {
    flex-shrink: 2;
}

/* Prevent content overflow */
.flex-item {
    flex-shrink: 0;
    min-width: 0; /* Allow content to wrap */
}
```

4.3 flex-basis

Sets the default size of an item before growing/shrinking.

4.3.1 Values

- auto - Size based on content (default)
- 0 - Start from zero, then grow
- Specific length (px,

4.3.2 Examples

```
/* Base size of 200px */
.flex-item {
    flex-basis: 200px;
}

/* Base on percentage */
.flex-item {
    flex-basis: 33.333%;
}

/* Zero base - grows from nothing */
.flex-item {
    flex-basis: 0;
    flex-grow: 1; /* Divides space equally */
}
```

4.4 flex Shorthand

Combines flex-grow, flex-shrink, and flex-basis.

```
/* flex: grow shrink basis */

/* Default */
.flex-item {
    flex: 0 1 auto;
}

/* Grow equally, don't shrink */
.flex-item {
    flex: 1 0 0;
}

/* Common pattern: grow equally */
.flex-item {
    flex: 1; /* = 1 1 0 */
}

/* Size but don't grow or shrink */
.flex-item {
    flex: 0 0 200px;
}
```

4.5 order

Changes the visual order of items without changing HTML.

```
.flex-item {
    order: 0; /* Default */
}
```

```
.flex-item-first {  
    order: -1; /* Appears first */  
}  
  
.flex-item-last {  
    order: 1; /* Appears last */  
}
```

4.6 align-self

Override align-items for a specific item.

```
.flex-container {  
    display: flex;  
    align-items: center; /* All items centered by default */  
}  
  
.flex-item-top {  
    align-self: flex-start; /* This item at top */  
}  
  
.flex-item-bottom {  
    align-self: flex-end; /* This item at bottom */  
}  
  
.flex-item-stretch {  
    align-self: stretch; /* This item stretches */  
}
```

Chapter 5

Practical Examples

5.1 Example 1: Navigation Bar

```
<nav class="navbar">
  <div class="logo">Logo</div>
  <div class="nav-links">
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Services</a>
    <a href="#">Contact</a>
  </div>
  <div class="user-menu">
    <button>Sign In</button>
  </div>
</nav>

.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 15px 30px;
  background: #333;
  color: white;
}

.logo {
  font-weight: bold;
  font-size: 20px;
}

.nav-links {
  display: flex;
  gap: 30px;
  flex-grow: 1;
  justify-content: center;
}

.nav-links a {
  text-decoration: none;
```



```
    color: white;
    transition: color 0.3s;
}

.nav-links a:hover {
    color: #3498db;
}

.user-menu {
    display: flex;
    gap: 10px;
}
```

5.2 Example 2: Card Layout

```
<div class="card-grid">
  <div class="card">
    
    <h3>Card Title</h3>
    <p>Card description text</p>
    <button>Learn More</button>
  </div>
  <!-- More cards -->
</div>

.card-grid {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
}

.card {
    display: flex;
    flex-direction: column;
    flex-basis: 250px;
    flex-grow: 1;
    background: white;
    border-radius: 8px;
    overflow: hidden;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.card img {
    width: 100%;
    height: 200px;
    object-fit: cover;
}

.card h3 {
    padding: 15px 15px 0;
    font-size: 18px;
}
```

```
.card p {
  padding: 0 15px 15px;
  color: #666;
  flex-grow: 1;
}

.card button {
  margin: 0 15px 15px;
  padding: 10px 20px;
  background: #3498db;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
```

5.3 Example 3: Form Layout

```
<form class="form">
  <div class="form-group">
    <label>Full Name</label>
    <input type="text" placeholder="Enter your name">
  </div>
  <div class="form-row">
    <div class="form-group">
      <label>Email</label>
      <input type="email" placeholder="Enter email">
    </div>
    <div class="form-group">
      <label>Phone</label>
      <input type="tel" placeholder="Enter phone">
    </div>
  </div>
  <button type="submit">Submit</button>
</form>

.form {
  display: flex;
  flex-direction: column;
  gap: 20px;
  max-width: 600px;
}

.form-group {
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.form-group label {
  font-weight: bold;
}
```

```
    font-size: 14px;
}

.form-group input {
    padding: 10px;
    border: 1px solid #ddd;
    border-radius: 4px;
    font-size: 14px;
}

.form-row {
    display: flex;
    gap: 20px;
}

.form-row .form-group {
    flex: 1;
}

.form button {
    padding: 12px 30px;
    background: #27ae60;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
}
```

5.4 Example 4: Footer Layout

```
<footer class="footer">
  <div class="footer-section">
    <h4>Company</h4>
    <ul>
      <li><a href="#">About Us</a></li>
      <li><a href="#">Careers</a></li>
      <li><a href="#">Blog</a></li>
    </ul>
  </div>
  <div class="footer-section">
    <h4>Support</h4>
    <ul>
      <li><a href="#">Help Center</a></li>
      <li><a href="#">Contact Us</a></li>
      <li><a href="#">FAQ</a></li>
    </ul>
  </div>
  <div class="footer-section">
    <h4>Legal</h4>
    <ul>
      <li><a href="#">Privacy</a></li>
```

```
        <li><a href="#">Terms</a></li>
        <li><a href="#">Cookies</a></li>
    </ul>
</div>
</footer>

.footer {
    display: flex;
    justify-content: space-around;
    gap: 40px;
    padding: 40px 20px;
    background: #2c3e50;
    color: white;
}

.footer-section {
    display: flex;
    flex-direction: column;
    gap: 15px;
}

.footer-section h4 {
    font-size: 16px;
    margin: 0;
}

.footer-section ul {
    display: flex;
    flex-direction: column;
    gap: 10px;
    list-style: none;
    padding: 0;
    margin: 0;
}

.footer-section a {
    color: #bdc3c7;
    text-decoration: none;
    transition: color 0.3s;
}

.footer-section a:hover {
    color: white;
}

/* Mobile responsive */
@media (max-width: 768px) {
    .footer {
        flex-direction: column;
        gap: 30px;
    }
}
```

Chapter 6

Responsive Flexbox

6.1 Mobile-First Approach

```
/* Mobile: Stack vertically */
.layout {
  display: flex;
  flex-direction: column;
  gap: 20px;
}

.sidebar {
  width: 100%;
  order: 2;
}

.main {
  width: 100%;
  order: 1;
}

/* Tablet: Side by side */
@media (min-width: 768px) {
  .layout {
    flex-direction: row;
    gap: 30px;
  }

  .sidebar {
    width: 250px;
    flex-shrink: 0;
    order: 2;
  }

  .main {
    flex-grow: 1;
    order: 1;
  }
}
```

6.2 Responsive Card Grid

```
/* All screens: Wrap on small screens */
.card-container {
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
}

.card {
  flex: 1 1 100%; /* 1 column on mobile */
  min-width: 200px;
}

/* Tablet: 2 columns */
@media (min-width: 768px) {
  .card {
    flex: 1 1 calc(50% - 7.5px);
  }
}

/* Desktop: 3+ columns */
@media (min-width: 1200px) {
  .card {
    flex: 1 1 calc(33.333% - 10px);
  }
}
```

Chapter 7

Common Issues & Solutions

7.1 Issue 1: Items Not Shrinking Below Content Size

Problem: Flex items won't shrink smaller than their content.

Solution: Add `min-width: 0` to allow text wrapping.

```
.flex-item {  
  flex-shrink: 1;  
  min-width: 0; /* Important! */  
}
```

7.2 Issue 2: Last Item Takes Extra Space

Problem: Last flex item stretches more than others.

Solution: Set explicit flex properties or use `margin-left: auto`.

```
/* Option 1: Consistent flex values */  
.flex-item {  
  flex: 1 1 0;  
}  
  
/* Option 2: Push to end */  
.flex-item:last-child {  
  margin-left: auto;  
}
```

7.3 Issue 3: Content Overflows Container

Problem: Long text or images overflow flexbox container.

Solution: Set `overflow: hidden` or use `text-overflow`.

```
.flex-item {  
  flex: 1;  
  overflow: hidden;  
}  
  
.flex-item h3 {
```

```
white-space: nowrap;
overflow: hidden;
text-overflow: ellipsis;
}
```

7.4 Issue 4: Unequal Item Sizes

Problem: Items in flex container have different sizes.

Solution: Use `flex-basis: 0` to start from equal base.

```
.flex-item {
  flex: 1 1 0; /* All grow equally from zero */
}
```


Chapter 8

Best Practices

8.1 Code Organization

```
/* GOOD: Clear separation of concerns */
.flex-container {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.flex-item {
  flex: 1;
}

/* BAD: Mixed concerns */
.wrapper {
  display: flex;
  background: blue;
  color: white;
  justify-content: center;
  align-items: center;
  padding: 20px;
  border-radius: 8px;
  flex-wrap: wrap;
}
```

8.2 Accessibility Tips

- Don't use `order` to hide important content
- Maintain logical source order in HTML
- Test keyboard navigation
- Ensure sufficient color contrast
- Use semantic HTML tags

8.3 Performance Tips

- Flexbox is very performant
- Prefer `gap` over margins
- Avoid complex nested flexboxes
- Use flexbox for layout, not micro-adjustments

8.4 When to Use Flexbox

- Navigation bars
- Card grids and layouts
- Form layouts
- Centering content
- Flexible menu systems
- One-dimensional layouts

8.5 When NOT to Use Flexbox

- Complex two-dimensional layouts (use Grid)
- Page-level structure (use Grid)
- When items need specific positioning

Chapter 9

Summary: Key Takeaways

9.1 Core Concepts

- Flexbox is one-dimensional layout
- Container properties control layout
- Item properties control individual items
- Main axis and cross axis are crucial
- Gap adds spacing between items

9.2 Essential Properties

Container	Item
display: flex	flex-grow
flex-direction	flex-shrink
justify-content	flex-basis
align-items	order
flex-wrap	align-self
gap	flex

9.3 Quick Reference

```
/* Basic flexbox setup */
.flex-container {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 20px;
}

/* Flexible items */
.flex-item {
  flex: 1;
}
```

```
/* Perfect centering */
.center-everything {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 300px;
}

/* Responsive layout */
@media (max-width: 768px) {
  .flex-container {
    flex-direction: column;
  }
}
```

9.4 Alignment Cheat Sheet

- `justify-content: center` - Center horizontally
- `align-items: center` - Center vertically
- Both together = Perfect center
- `justify-content: space-between` - Space items
- `justify-content: space-around` - Space with margins
- `justify-content: flex-end` - Right align
- `align-items: flex-end` - Bottom align

9.5 Key Rules

1. Always use flexbox for one-dimensional layouts
2. Container properties affect all items
3. Item properties override container for that item
4. `flex: 1` makes items grow equally
5. `gap` is better than margins
6. Use `flex-wrap: wrap` for responsive grids
7. `flex-basis: 0` for equal sizing
8. Test on different screen sizes
9. Maintain semantic HTML structure
10. Flexbox is performant - use freely